

Task 1 — Create MariaDB Database on EC2

Objective

To launch an Amazon EC2 instance and install/configure MariaDB Server on it for database operations.

Implementation Steps

1. Launched an **Amazon EC2 instance** with the following configuration:
 - a. **Instance Name:** MariaDB-EC2
 - b. **Instance Type:** t3.micro
 - c. **AMI:** Amazon Linux 2023
 - d. **VPC:** CLB-VPC
 - e. **Subnet:** Public Subnet 1
 - f. **Private IP:** 10.10.1.124
 - g. **Public IP:** 32.192.179.22
2. Created and attached the security group:
 - a. **Security Group:** MariaDB-EC2-SG
 - b. **Inbound:** SSH (TCP 22)
 - c. **Source:** My IP
 - d. **Outbound:** All traffic allowed
3. Connected to the EC2 instance using **SSH through Git Bash**.
4. Installed MariaDB Server using the Amazon Linux package manager.
5. Enabled MariaDB to start automatically when the EC2 instance starts.
6. Started the MariaDB service.
7. Verified the MariaDB service using:

```
sudo systemctl status mariadb
```

8. Verified that the service was successfully running. The output showed:

Active: active (running)

Status: "Taking your SQL requests now..."

EC2 > Instances > Launch an instance

Allow tags in metadata

Info

Select

User data - optional

Info

Upload a file with your user data or enter it in the field.

Choose file

```
#!/bin/bash

dnf update -y
dnf install mariadb105-server -y

systemctl enable mariadb
systemctl start mariadb
```

☐ User data has already been base64 encoded

EC2 > Instances > i-0ad208fe7ef4219a4

EC2

Dashboard

AWS Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Instance summary for i-0ad208fe7ef4219a4 (MariaDB-EC2)

Updated less than a minute ago

Instance ID

i-0ad208fe7ef4219a4

IPv6 address

-

Hostname type

IP name: ip-10-10-1-124.ec2.internal

Answer private resource DNS name

-

Auto-assigned IP address

32.192.179.22 [Public IP]

IAM role

-

IMDSv2

Required

Public IPv4 address

32.192.179.22 | open address

Instance state

Running

Private IP DNS name (IPv4 only)

ip-10-10-1-124.ec2.internal

Instance type

t3.micro

VPC ID

vpc-001568701c18b6d49 (CLB-VPC)

Subnet ID

subnet-0df87df2dfb262341 (Public Subnet 1)

Instance ARN

arn:aws:ec2:us-east-1:331116047691:instance/i-0ad208fe7ef4219a4

Private IPv4 addresses

10.10.1.124

Public DNS

-

Elastic IP addresses

-

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendation

Auto Scaling Group name

-

Managed

false

```

ec2-user@ip-10-10-1-124:~
asifs@Asif_Shaik MINGW64 ~
$ cd ~/downloads

asifs@Asif_Shaik MINGW64 ~/downloads (master)
$ ssh -i "loadbalancer.pem" ec2-user@32.192.179.22
The authenticity of host '32.192.179.22 (32.192.179.22)' can't be established.
ED25519 key fingerprint is SHA256:uN0dEcn3M/Rj2XbIZls1jd6eg4Cxc6rqF+NKtbSqe0Q.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '32.192.179.22' (ED25519) to the list of known hosts.

      #
      ##_
      #####
      #####\
      \###|
      \##/
      V~'  '---> https://aws.amazon.com/linux/amazon-linux-2023
      /m/'

[ec2-user@ip-10-10-1-124 ~]$ mariadb --version
mariadb Ver 15.1 Distrib 10.5.29-MariaDB, for Linux (x86_64) using EditLine wrapper

[ec2-user@ip-10-10-1-124 ~]$ sudo systemctl status mariadb
● mariadb.service - MariaDB 10.5 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: >
   Active: active (running) since Wed 2026-09-16 10:40:42 UTC; 14min ago
     Docs: man:mariadb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 4541 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, >
   Process: 4631 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.serv>
   Process: 8270 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited>
  Main PID: 7961 (mariabdb)
    Status: "Taking your SQL requests now..."
     Tasks: 8 (limit: 1059)
    Memory: 66.1M
       CPU: 634ms
    CGroup: /system.slice/mariadb.service
            └─7961 /usr/libexec/mariabdb --basedir=/usr

Sep 16 10:40:42 ip-10-10-1-124.ec2.internal mariadb-prepare-db-dir[6517]: The s>
Sep 16 10:40:42 ip-10-10-1-124.ec2.internal mariadb-prepare-db-dir[6517]: you n>
Sep 16 10:40:42 ip-10-10-1-124.ec2.internal mariadb-prepare-db-dir[6517]: After>
Sep 16 10:40:42 ip-10-10-1-124.ec2.internal mariadb-prepare-db-dir[6517]: able>
Sep 16 10:40:42 ip-10-10-1-124.ec2.internal mariadb-prepare-db-dir[6517]: See t>
Sep 16 10:40:42 ip-10-10-1-124.ec2.internal mariadb-prepare-db-dir[6517]: Pleas>
Sep 16 10:40:42 ip-10-10-1-124.ec2.internal mariadb-prepare-db-dir[6517]: The l>

```

Result

MariaDB Server 10.5 was successfully installed and configured on the MariaDB-EC2 instance. The MariaDB service is running successfully and is enabled to start automatically after an EC2 reboot.

Conclusion

Successfully completed **Task 1 – Create MariaDB Database on EC2**. The EC2 instance is ready for the next step, which is to create a database, table, and insert dummy data.

Task 2: Create Database and Verify MariaDB

Objective

To create a database in MariaDB and verify that the database was successfully created.

Steps Performed

1. Connected to the MariaDB server from the EC2 instance using:

```
sudo mariadb
```

2. Created the database named:

```
CREATE DATABASE devopsdb;
```

3. Verified the available databases using:

```
SHOW DATABASES;
```

4. Confirmed that devopsdb was displayed in the list of databases.

```
[ec2-user@ip-10-10-1-124 ~]$ sudo mariadb
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 3
Server version: 10.5.29-MariaDB MariaDB Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE devopsdb;
Query OK, 1 row affected (0.000 sec)

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| devopsdb |
| information_schema |
| mysql |
| performance_schema |
+-----+
4 rows in set (0.001 sec)

MariaDB [(none)]> USE devopsdb;
Database changed
MariaDB [devopsdb]> CREATE TABLE employees (
->     id INT PRIMARY KEY AUTO_INCREMENT,
->     name VARCHAR(100),
->     role VARCHAR(100),
->     salary INT
-> );
Query OK, 0 rows affected (0.008 sec)

MariaDB [devopsdb]> INSERT INTO employees (name, role, salary) VALUES
-> ('Asif', 'DevOps Engineer', 60000),
-> ('Rahul', 'Cloud Engineer', 55000),
-> ('Kiran', 'Linux Administrator', 45000),
-> ('Arun', 'Software Engineer', 50000);
Query OK, 4 rows affected (0.003 sec)
Records: 4  Duplicates: 0  Warnings: 0

MariaDB [devopsdb]> SELECT * FROM employees;
+----+-----+-----+-----+
| id | name | role | salary |
+----+-----+-----+-----+
| 1 | Asif | DevOps Engineer | 60000 |
| 2 | Rahul | Cloud Engineer | 55000 |
| 3 | Kiran | Linux Administrator | 45000 |
| 4 | Arun | Software Engineer | 50000 |
+----+-----+-----+-----+
4 rows in set (0.000 sec)

MariaDB [devopsdb]> EXIT;
Bye
```

Result

The devopsdb database was successfully created and verified using the SHOW DATABASES; command.

Evidence

Screenshot of the MariaDB terminal showing the SHOW DATABASES; output with devopsdb.

Conclusion

Successfully created and verified the devopsdb database on the MariaDB server running on the EC2 instance. The database is ready for creating tables and inserting dummy data.

Task 3 — Take Backup of the Database on EC2

Objective

To take a backup of the devopsdb MariaDB database containing the dummy employee data.

Steps Performed

1. Exited the MariaDB console and returned to the EC2 terminal.
2. Created a SQL dump of the devopsdb database using:

```
sudo mysqldump devopsdb > ~/devopsdb_backup.sql
```

3. Verified that the backup file was created:

```
ls -lh ~/devopsdb_backup.sql
```

Output confirmed:

```
-rw-r--r--. 1 ec2-user ec2-user 2.2K Sep 16 11:16 /home/ec2-user/devopsdb_backup.sql
```

4. Checked the contents of the backup file:

```
sudo head -20 ~/devopsdb_backup.sql
```

The output showed MariaDB dump information and the table structure for the employees table.

5. Verified the number of lines in the backup:

```
sudo wc -l ~/devopsdb_backup.sql
```

Output:

```
54 /home/ec2-user/devopsdb_backup.sql
```

Result

The devopsdb database was successfully backed up to:

```
/home/ec2-user/devopsdb_backup.sql
```

The backup file contains the database structure and data required to restore the database.

```

[ec2-user@ip-10-10-1-124 ~]$ sudo mysqldump devopsdb > ~/devopsdb_backup.sql
[ec2-user@ip-10-10-1-124 ~]$ ls -lh ~/devopsdb_backup.sql
-rw-r--r--. 1 ec2-user ec2-user 2.2K Sep 16 11:16 /home/ec2-user/devopsdb_backup.sql
[ec2-user@ip-10-10-1-124 ~]$ sudo head -20 ~/devopsdb_backup.sql
/*!9999999\~ enable the sandbox mode */
-- MariaDB dump 10.19 Distrib 10.5.29-MariaDB, for Linux (x86_64)
--
-- Host: localhost    Database: devopsdb
--
-- Server version      10.5.29-MariaDB

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
/*!40014 SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;

--
-- Table structure for table `employees`
[ec2-user@ip-10-10-1-124 ~]$ sudo wc -l ~/devopsdb_backup.sql
54 /home/ec2-user/devopsdb_backup.sql
[ec2-user@ip-10-10-1-124 ~]$ ls -lh ~/devopsdb_backup.sql
-rw-r--r--. 1 ec2-user ec2-user 2.2K Sep 16 11:16 /home/ec2-user/devopsdb_backup.sql

```

Conclusion

Task 3 was successfully completed. The MariaDB database backup is ready to be used for migration to Amazon RDS in the next task.

Task 4 — Launch MariaDB RDS instance.

Objective

To launch a MariaDB database instance on Amazon RDS that will be used as the target database for migrating the MariaDB database from EC2.

Steps Performed

1. Open Amazon RDS

- Logged in to the AWS Management Console.
- Opened Amazon RDS.
- Selected Databases.
- Clicked Create database.

2. Select Database Engine

- Selected Standard create / Full configuration.
- Selected MariaDB as the database engine.

3. Configure Database

- DB Identifier: mariadb-rds-devops
- Master Username: admin
- Selected Self managed credentials.
- Configured the database using the available low-cost instance option:
 - Instance class: db.t4g.micro

4. Configure Connectivity

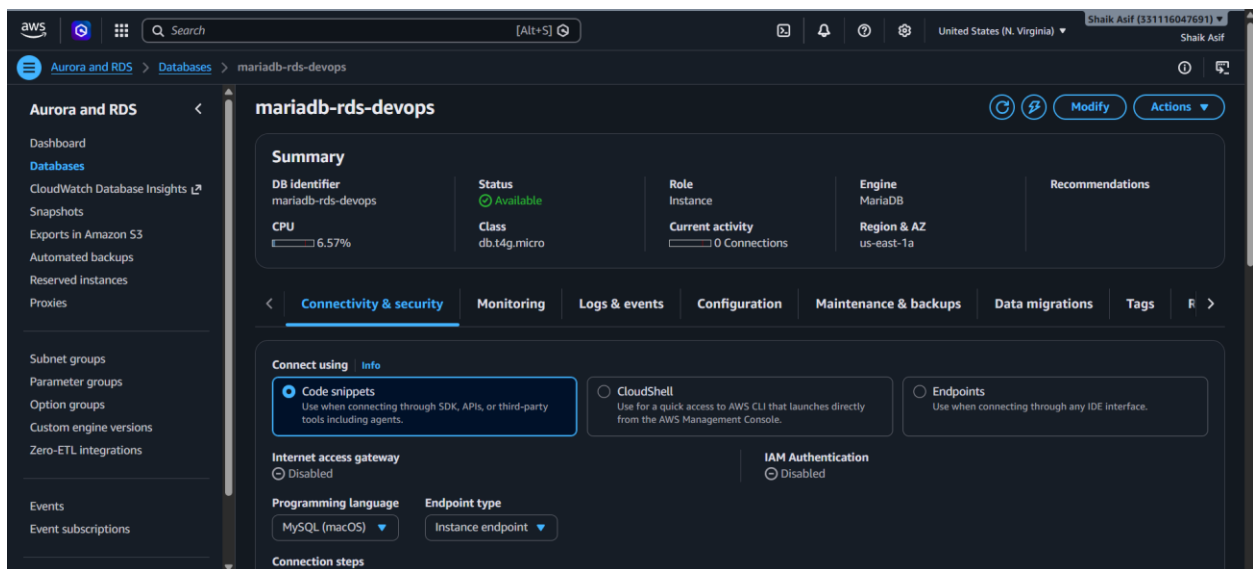
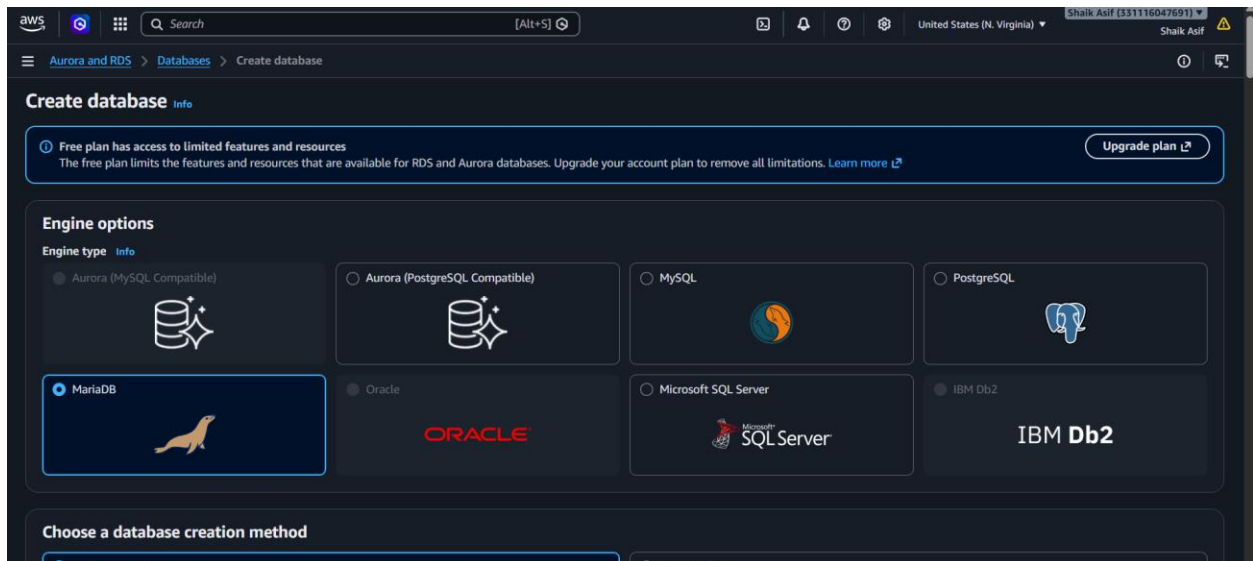
- VPC: CLB-VPC
- Network type: IPv4
- Public access: No
- Created a new security group:
 - Security Group: MariaDB-RDS-SG
- Did not configure an RDS Proxy.

5. Database Configuration

- Initial Database Name: devopsdb
- Multi-AZ: Disabled for this task.
- Encryption: Enabled.
- Automated Backups: Enabled.
- Backup Retention: 1 day.
- Deletion Protection: Disabled.

6. Create the RDS Instance

- Reviewed the configuration.
- Clicked Create database.
- The MariaDB RDS instance was successfully created.



Result

The MariaDB RDS instance **mariadb-rds-devops** was successfully created in the CLB-VPC.

The RDS instance will be used as the **target database** for migrating the devopsdb database from the MariaDB EC2 server.

Conclusion

Successfully launched a **MariaDB database using Amazon RDS** with private connectivity, encryption, automated backup, and the devopsdb initial database. The RDS instance is now ready for the database migration activity in the next task.

Task 5 — Migrate MariaDB from EC2 to RDS

Objective

Migrate the devopsdb MariaDB database from the EC2 server to the Amazon RDS MariaDB instance and verify that all data is successfully migrated.

Step 1: Connect to RDS MariaDB

From the MariaDB-EC2 instance, I connected to the RDS MariaDB instance using SSL/TLS.

Command

```
mysql -h mariadb-rds-devops.cab66qsqy9vb.us-east-1.rds.amazonaws.com -u admin -p --ssl
```

Output

Welcome to the MariaDB monitor. Commands end with ; or \g.

Your MariaDB connection id is 64

Server version: 11.8.8-MariaDB-log managed by AWS

This confirmed that the EC2 instance could securely connect to the RDS MariaDB instance.

Step 2: Check Databases on RDS

Command

```
SHOW DATABASES;
```

Initial Output

```
+-----+
| Database |
+-----+
| information_schema |
| innodb |
| mysql |
| performance_schema |
```

```
| sys          |
+-----+
5 rows in set
```

The devopsdb database was not present on RDS.

Step 3: Create the Database on RDS

Command

```
CREATE DATABASE devopsdb;
```

Output

Query OK, 1 row affected

I then verified the database.

Command

```
SHOW DATABASES;
```

Output

```
+-----+
| Database          |
+-----+
| devopsdb          |
| information_schema |
| innodb            |
| mysql             |
| performance_schema |
| sys               |
+-----+
6 rows in set
```

Step 4: Restore the EC2 Backup into RDS

The database backup created during Task 3 was:

```
~/devopsdb_backup.sql
```

I restored this backup into the RDS devopsdb database.

Command

```
mysql --ssl -h mariadb-rds-devops.cab66qsqy9vb.us-east-1.rds.amazonaws.com -u admin -p  
devopsdb < ~/devopsdb_backup.sql
```

Output

```
[ec2-user@ip-10-10-1-124 ~]$
```

No error was displayed, indicating that the backup was successfully imported into RDS.

Step 5: Verify the Migrated Table

I connected to the RDS database:

```
mysql --ssl -h mariadb-rds-devops.cab66qsqy9vb.us-east-1.rds.amazonaws.com -u admin -p  
devopsdb
```

Then checked the tables.

Command

```
SHOW TABLES;
```

Output

```
+-----+  
| Tables_in_devopsdb |  
+-----+  
| employees          |  
+-----+  
1 row in set
```

Step 6: Verify the Migrated Data

Command

```
SELECT * FROM employees;
```

Output

```
+----+-----+-----+-----+
| id | name | role          | salary |
+----+-----+-----+-----+
| 1 | Asif | DevOps Engineer | 60000 |
| 2 | Rahul | Cloud Engineer  | 55000 |
| 3 | Kiran | Linux Administrator | 45000 |
| 4 | Arun | Software Engineer | 50000 |
+----+-----+-----+-----+
```

4 rows in set

```
[ec2-user@ip-10-10-1-124 ~]$ mysql --ssl -h mariadb-rds-devops.cab66qsqy9vb.us-east-1.rds.amazonaws.com -u admin -p devopsdb < ~/devopsdb_backup.sql
Enter password:
[ec2-user@ip-10-10-1-124 ~]$ mysql --ssl -h mariadb-rds-devops.cab66qsqy9vb.us-east-1.rds.amazonaws.com -u admin -p devopsdb
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 73
Server version: 11.8.8-MariaDB-log managed by https://aws.amazon.com/rds/

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [devopsdb]> SHOW TABLES;
+-----+
| Tables_in_devopsdb |
+-----+
| employees           |
+-----+
1 row in set (0.001 sec)

MariaDB [devopsdb]> SELECT * FROM employees;
+----+-----+-----+-----+
| id | name | role          | salary |
+----+-----+-----+-----+
| 1 | Asif | DevOps Engineer | 60000 |
| 2 | Rahul | Cloud Engineer  | 55000 |
| 3 | Kiran | Linux Administrator | 45000 |
| 4 | Arun | Software Engineer | 50000 |
+----+-----+-----+-----+
4 rows in set (0.001 sec)
```

Result

The devopsdb database was successfully migrated from the MariaDB database running on EC2 to Amazon RDS MariaDB.

The employees table and all 4 dummy records were successfully restored and verified on RDS.

Conclusion

The MariaDB database migration from EC2 to RDS was completed successfully using mysqldump backup and restore. The RDS connection was established securely using SSL/TLS, and the migrated data was verified successfully.

Task 6 - Install MySQL DB on EC2.

Objective

To install and configure MySQL 8.4 Community Server on an Amazon Linux EC2 instance and verify that the MySQL service is running successfully.

Steps Performed

Step 1: Install MySQL 8.4 Community Repository

Installed the MySQL 8.4 Community repository:

```
sudo dnf install https://dev.mysql.com/get/mysql84-community-release-el9-1.noarch.rpm -y
```

The MySQL 8.4 LTS repository was successfully enabled.

Step 2: Install MySQL Server

Installed the MySQL Community Server:

```
sudo dnf install mysql-community-server -y
```

MySQL 8.4.11 and its required packages were installed successfully.

Step 3: Resolve MariaDB Package Conflict

The EC2 instance had existing MariaDB packages that conflicted with MySQL.

The conflicting MariaDB packages were removed before installing MySQL successfully.

Step 4: Start MySQL Service

Attempted to start MySQL:

```
sudo systemctl start mysqld
```

Initially, the service failed because old MariaDB database files were present in `/var/lib/mysql`.

Step 5: Check MySQL Error Logs

Checked the MySQL log:

```
sudo tail -50 /var/log/mysqld.log
```

The logs showed InnoDB corruption and incompatible existing data files, including `ibdata1`.

Step 6: Remove Old Database Files

Since this was a fresh lab environment and there was no important database data, the old database files were removed.

Used:

```
sudo find /var/lib/mysql -mindepth 1 -maxdepth 1 -exec rm -rf {} +
```

Verified the directory:

```
sudo ls -la /var/lib/mysql
```

The directory was empty except for . and ...

Step 7: Initialize MySQL

Initialized the MySQL 8.4 data directory:

```
sudo mysqld --initialize --user=mysql
```

The initialization completed successfully and created the required MySQL database files.

Step 8: Start MySQL Service

Started MySQL:

```
sudo systemctl start mysqld
```

Verified the service status:

```
sudo systemctl status mysqld --no-pager
```

The service showed:

Active: active (running)

Step 9: Retrieve Temporary Root Password

Retrieved the temporary password generated during initialization:

```
sudo grep 'temporary password' /var/log/mysqld.log
```

A temporary root password was generated for root@localhost.

Step 10: Login to MySQL

Logged into MySQL using:


```
mysql -u root -p
```

The temporary password was provided when prompted.

Successfully reached the MySQL prompt:

```
mysql>
```

Step 11: Set Permanent Root Password

Changed the temporary root password using:

```
ALTER USER 'root'@'localhost' IDENTIFIED BY 'MySQL@12345';
```

The password was successfully changed.

Step 12: Verify MySQL Login

Exited MySQL:

```
exit;
```

Logged in again using:

```
mysql -u root -p
```

The new password was accepted and the MySQL prompt was displayed successfully.

MySQL version was also verified using:

```
SELECT VERSION();
```

The installed version was MySQL 8.4.

aws

Search

[Alt+S]

United States (N. Virginia)

Shaik Asif (33116047691)

Shaik Asif

EC2

Instances

I-05f1e929339639129

EC2

Dashboard

AWS Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Capacity Manager

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Instance summary for i-05f1e929339639129 (MySQL-EC2)

Info

Updated less than a minute ago

Instance ID

I-05f1e929339639129

Public IPv4 address

44.203.62.73 | open address

Private IPv4 addresses

10.10.1.192

IPv6 address

-

Instance state

Running

Public DNS

-

Hostname type

IP name: ip-10-10-1-192.ec2.internal

Private IP DNS name (IPv4 only)

ip-10-10-1-192.ec2.internal

Answer private resource DNS name

-

Instance type

t3.micro

Elastic IP addresses

-

Auto-assigned IP address

44.203.62.73 [Public IP]

VPC ID

vpc-001568701c18b6d49 (CLB-VPC)

AWS Compute Optimizer finding

Opt-in to AWS Compute Optimizer for recommendation

Learn more

IAM role

-

Subnet ID

subnet-0df87df2dfb262341 (Public Subnet 1)

Auto Scaling Group name

-

IMDSv2

Required

Instance ARN

arn:aws:ec2:us-east-1:33116047691:instance/i-05f1e929339639129

Managed

false

```
[ec2-user@ip-10-10-1-192 ~]$ sudo grep 'temporary password' /var/log/mysql.log
2026-09-16T12:43:39.710440Z 6 [Note] [MY-010454] [Server] A temporary password is generated for root@localhost: q2qdPghFri1r
[ec2-user@ip-10-10-1-192 ~]$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.4.11

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> ALTER USER 'root'@'localhost' IDENTIFIED BY 'MySQL@12345';
Query OK, 0 rows affected (0.01 sec)

mysql> exit;
Bye
```

```
Bye
[ec2-user@ip-10-10-1-192 ~]$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 9
Server version: 8.4.11 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT VERSION();
+-----+
| VERSION() |
+-----+
| 8.4.11    |
+-----+
1 row in set (0.00 sec)
```

Result

MySQL 8.4 Community Server was successfully installed and configured on the Amazon Linux EC2 instance.

- MySQL Server: 8.4
- Service: Running
- Root login: Successful
- Data directory: Successfully initialized
- Permanent root password: Configured

Conclusion

MySQL 8.4 was successfully installed, initialized, configured, and verified on the EC2 instance. The initial startup issue caused by old MariaDB data files was resolved by clearing the old

database data and reinitializing the MySQL data directory. The MySQL service is now running successfully and the root user can log in with the newly configured password.

Task 7: Launch MySQL RDS Image

Objective

To launch a managed MySQL database using Amazon RDS with the required configuration.

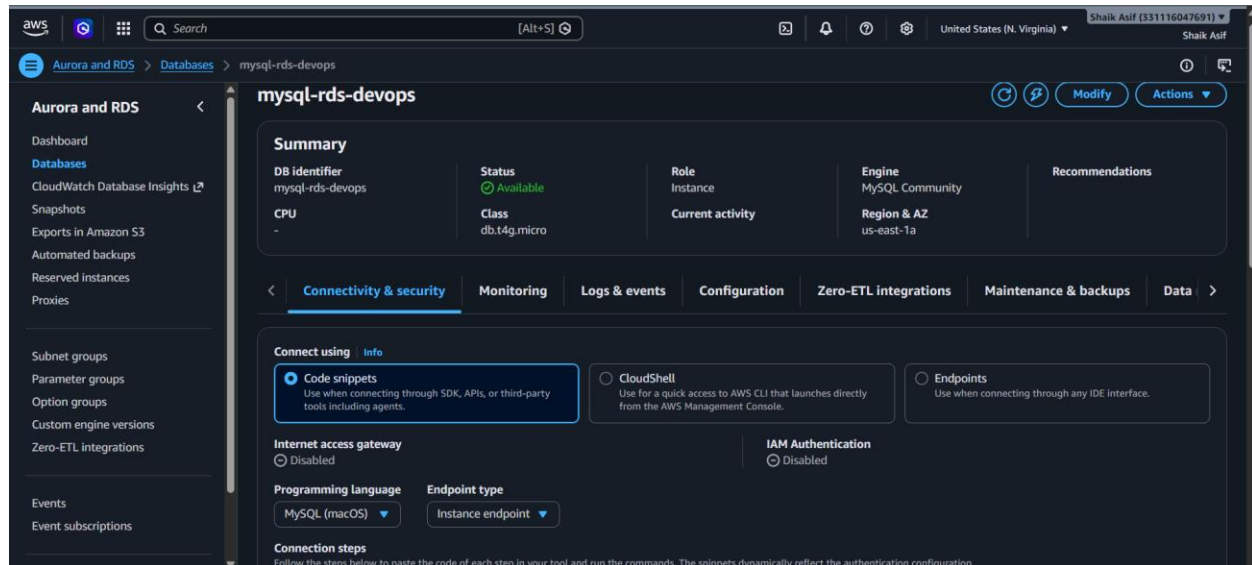
Steps Performed

1. Opened AWS Console → RDS → Databases.
2. Clicked Create database.
3. Selected:
 - a. Creation method: Full configuration
 - b. Engine type: MySQL
 - c. Edition: MySQL Community
 - d. Engine version: MySQL 8.4.9
4. Selected the Free tier template.
5. Selected:
 - a. Deployment: Single-AZ DB instance
 - b. DB instance identifier: mysql-rds-devops
6. Configured the master credentials:
 - a. Master username: admin
 - b. Selected Auto generate a password.
 - c. The generated password was kept securely and was not shared.
7. Configured the DB instance:
 - a. Instance class: db.t4g.micro
 - b. Storage: 20 GiB
 - c. Storage type: General Purpose SSD
8. Under connectivity, selected the default VPC:
 - a. VPC: vpc-048b13dd0eb01d950
 - b. DB subnet group: default

- c. No EC2 compute resource was connected.
9. Created a new security group:
 - a. Security group: mysql-rds-sg
 - b. Configured MySQL inbound access on TCP port 3306 from the required IP address.
10. Kept the database deployment as Single-AZ and allowed AWS to select the Availability Zone.
11. Configured monitoring with Database Insights – Standard and kept Enhanced Monitoring disabled.
12. Kept RDS Proxy disabled/not created.
13. Disabled Delete Protection for the practice database.
14. Reviewed all the settings and clicked Create database.
15. Waited for the RDS instance to finish provisioning.
16. Verified that the database status changed to:

Available

17. Verified the final RDS details:
 - DB identifier: mysql-rds-devops
 - Engine: MySQL Community
 - Status: Available
 - Instance class: db.t4g.micro
 - Region: N. Virginia (us-east-1)
 - Availability Zone: us-east-1a
 - Port: 3306
 - Endpoint:
mysql-rds-devops.cab66qsqy9vb.us-east-1.rds.amazonaws.com



Result

The MySQL RDS instance was successfully launched and reached the Available state. The database is configured to accept MySQL connections through port 3306 according to the configured security group.

Conclusion

The MySQL RDS image was successfully launched and configured in AWS using Amazon RDS. The database instance mysql-rds-devops is available and ready for use.

Task 8 - Configure multi-AZ

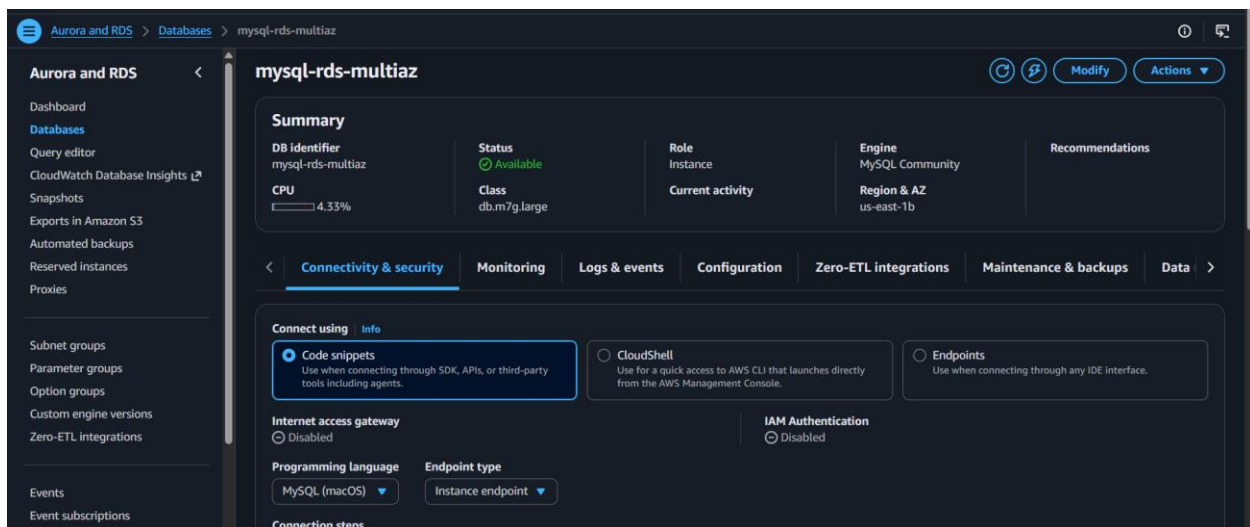
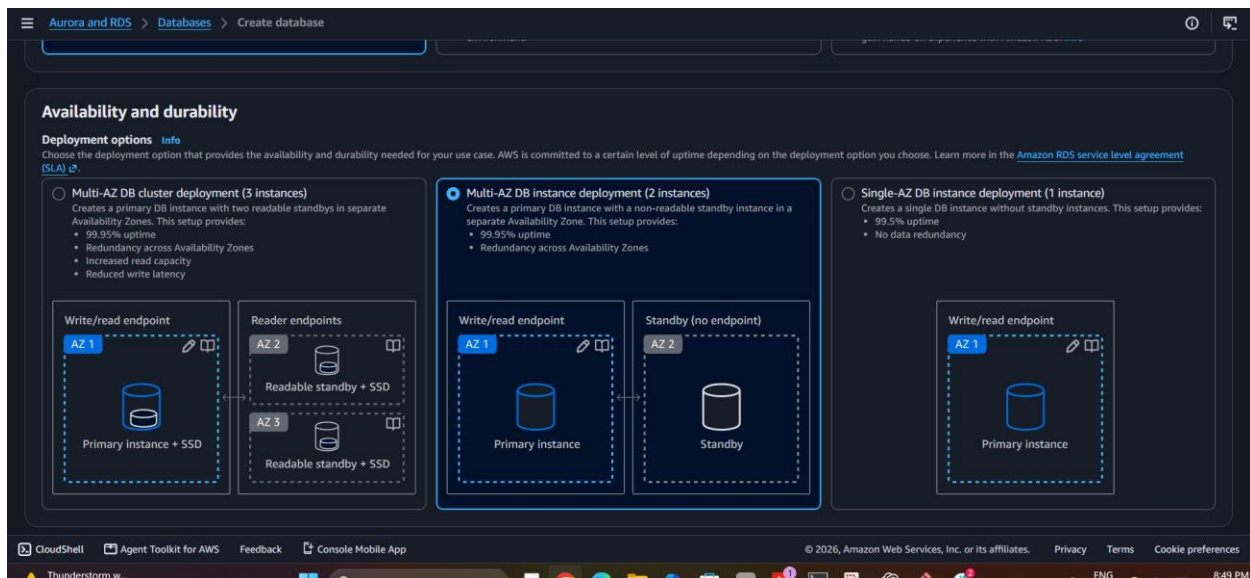
Objective

To configure an Amazon RDS MySQL database with Multi-AZ deployment for high availability and automatic failover.

Steps Implemented

1. Logged in to the AWS Management Console.
2. Opened Amazon RDS service.
3. Navigated to RDS → Databases and selected Create database.
4. Selected MySQL as the database engine.
5. Selected the appropriate database template and configured the required database credentials.
6. Under Availability & durability, selected Multi-AZ DB instance deployment.

7. Selected the Default VPC and the default DB subnet group.
8. Configured the connectivity settings and selected the required VPC security group.
9. Disabled unnecessary additional monitoring and optional services for the practice environment.
10. Reviewed the database configuration and estimated cost.
11. Created the RDS database.
12. Waited until the database status changed to Available.
13. Opened the database Configuration details and verified the Multi-AZ deployment and Availability Zone information.
14. Captured screenshots as evidence of the successful Multi-AZ configuration.



Aurora and RDS

Databases

mysql-rds-multiaz

Databases

Query editor

CloudWatch Database Insights

Snapshots

Exports in Amazon S3

Automated backups

Reserved instances

Proxies

Subnet groups

Parameter groups

Option groups

Custom engine versions

Zero-ETL integrations

Events

Event subscriptions

Instance

Configuration

DB instance ID
mysql-rds-multiaz

Engine version
8.4.9

RDS Extended Support
Disabled

DB name
-

License model
General Public License

Option groups
default:mysql-8-4 In sync

Amazon Resource Name (ARN)
 arn:aws:rds:us-east-1:84662419859:4:db:mysql-rds-multiaz

Resource ID
db-FCBZAWDTJWCIZISPRHKLEPODGE

Created time

Instance class

Instance class
db.m7g.large

vCPU
2

RAM
8 GB

Availability

Master username
admin

Master password

IAM DB authentication
Not enabled

Multi-AZ
Yes

Secondary Zone
us-east-1a

Primary storage

Storage operation status
Initializing

Storage operation progress
0%

Encryption
Enabled

AWS KMS key
[aws/rds](#)

Storage type
Provisioned IOPS SSD (io2)

Storage
400 GiB

Provisioned IOPS
3000 IOPS

Storage throughput
-

Storage autoscaling
Enabled

Monitoring

Monitoring type
Database Insights - Advanced

Database counter metrics
Enabled

Retention period
465 days

AWS KMS key
[aws/rds](#)

Enhanced Monitoring
Enabled

Granularity
60 seconds

Monitoring role
 arn:aws:iam:84662419859:role/rds-monitoring-role

Instance

Configuration

DB instance ID
mysql-rds-multiaz

Engine version
8.4.9

RDS Extended Support
Disabled

DB name
-

License model
General Public License

Option groups
default:mysql-8-4 In sync

Amazon Resource Name (ARN)
 arn:aws:rds:us-east-1:84662419859:4:db:mysql-rds-multiaz

Resource ID
db-FCBZAWDTJWCIZISPRHKLEPODGE

Created time

Instance class

Instance class
db.m7g.large

vCPU
2

RAM
8 GB

Availability

Master username
admin

Master password

IAM DB authentication
Not enabled

Multi-AZ
Yes

Secondary Zone
us-east-1a

Result

The RDS MySQL database was successfully created with **Multi-AZ deployment enabled** and the configuration was verified from the RDS console.

Conclusion

Multi-AZ provides **high availability** by maintaining a standby database in a different Availability Zone. In case of a failure of the primary database, Amazon RDS can perform automatic failover to the standby instance, improving database availability and reliability.

Task 9 – Take Backup of DB and Restore the DB

Objective

To create a backup of the MySQL RDS database and restore the backup as a new RDS database instance.

Steps

Step 1: Create a Manual DB Snapshot

1. Open AWS Console → RDS → Databases.
2. Select the RDS instance:
mysql-rds-devops
3. Click Actions.
4. Select Take snapshot.
5. Enter the snapshot identifier:
mysql-rds-devops-backup
6. Click Take snapshot.
7. Wait until the snapshot status changes to Available.

Step 2: Restore the DB from Snapshot

1. Go to RDS → Snapshots.
2. Select:
mysql-rds-devops-backup
3. Click Actions → Restore snapshot.
4. In DB instance identifier, enter:
mysql-rds-devops-restored
5. Keep the required configuration settings.

- Click Restore DB instance.
- Wait until the restored RDS instance status becomes Available.

Resources Created

Resource	Name
Original RDS	mysql-rds-devops
Manual Snapshot	mysql-rds-devops-backup
Restored RDS	mysql-rds-devops-restored

The screenshot shows the AWS Management Console interface for the 'mysql-rds-devops-backup' snapshot. The left sidebar contains navigation links for Aurora and RDS, including Dashboard, Databases, CloudWatch Database Insights, Snapshots, Exports in Amazon S3, Automated backups, Reserved instances, Proxies, Subnet groups, Parameter groups, Option groups, Custom engine versions, Zero-ETL integrations, Events, and Event subscriptions. The main content area displays the 'Details' of the snapshot, organized into three columns:

Details		
ARN arn:aws:rds:us-east-1:331116047691:snapshot:mysql-rds-devops-backup	Option group default:mysql-8-4	VPC vpc-048b13dd0eb01d950
Instance/Cluster Name mysql-rds-devops	Zone us-east-1a	Status Creating
Master username admin	KMS key ID e76c0d7f-b45d-46ad-8c6b-889e99e631cc	Storage type General Purpose SSD (gp2)
DB snapshot name mysql-rds-devops-backup	Source region N/A	DB storage 20 GiB
Snapshot type manual	Snapshot Creation Time September 16, 2026, 21:37 (UTC+05:30)	IOPS -
DB engine mysql	Instance/Cluster Creation September 16, 2026, 21:09 (UTC+05:30)	Storage throughput 0
DB engine version 8.4.9	Backup retention period 1 day	Port 3306
License model general-public-license	Backup window 04:58-05:28	Time zone -
		Dedicated Log Volume Turned off

The screenshot shows the 'Restore snapshot' wizard in the AWS Management Console. The top navigation bar includes the AWS logo, search bar, and user information (Shaik Asif). The main content area is titled 'Restore snapshot' and includes a warning about the free plan's limitations. Below this, the 'DB instance settings' section shows the 'DB engine' set to 'MySQL Community'. The 'Availability and durability' section offers three deployment options: Multi-AZ DB cluster deployment (3 instances), Multi-AZ DB instance deployment (2 instances), and Single-AZ DB instance deployment (1 instance). The Single-AZ option is selected.

Free plan has access to limited features and resources
The free plan limits the features and resources that are available for RDS and Aurora databases. Upgrade your account plan to remove all limitations. [Learn more](#)

Upgrade plan

DB instance settings

DB engine
MySQL Community

☐ **Enable RDS Extended Support** [Info](#)
Amazon RDS Extended Support is a [paid offering](#). By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the [RDS for MySQL documentation](#).

Availability and durability

Deployment options [Info](#)
Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. Learn more in the [Amazon RDS service level agreement \(SLA\)](#).

- ☐ **Multi-AZ DB cluster deployment (3 instances)**
Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:
 - 99.95% uptime
 - Read-only standby instances in separate Availability Zones
- ☐ **Multi-AZ DB instance deployment (2 instances)**
Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:
 - 99.95% uptime
 - Read-only standby instance in separate Availability Zone
- ☒ **Single-AZ DB instance deployment (1 instance)**
Creates a single DB instance without standby instances. This setup provides:
 - 99.5% uptime
 - No data redundancy

Result

The MySQL RDS database was successfully backed up using a manual snapshot, and the snapshot was successfully restored as a new RDS database instance named mysql-rds-devops-restored.

Conclusion

The backup and restore process was successfully completed. This demonstrates how an RDS database can be backed up using a manual snapshot and recovered into a separate RDS instance when required.

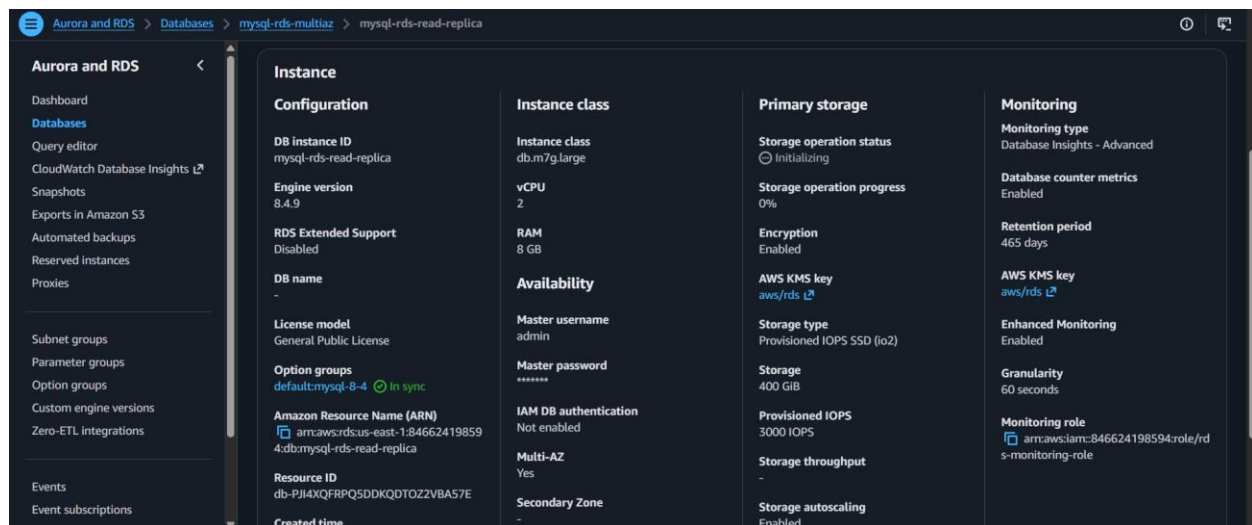
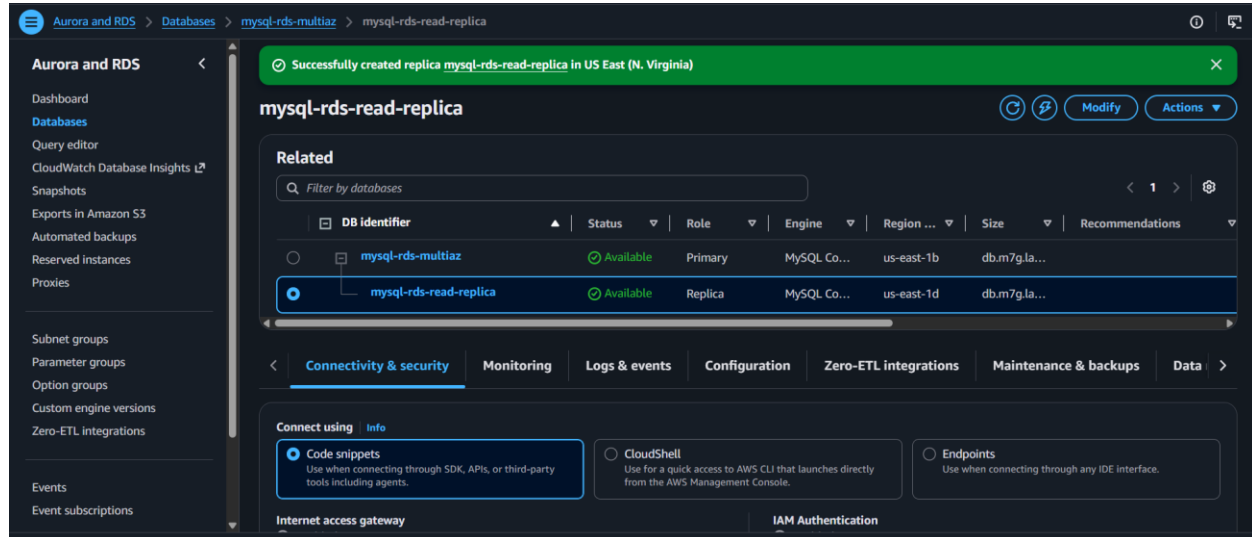
Task 10 - Create read replica.

Objective

To create an Amazon RDS Read Replica from the primary MySQL database to support read-heavy workloads and improve database performance and scalability.

Steps Implemented

1. Logged in to the AWS Management Console.
2. Opened the **Amazon RDS** service.
3. Navigated to **RDS → Databases**.
4. Selected the primary MySQL RDS database created for the Multi-AZ configuration.
5. Clicked **Actions → Create read replica**.
6. Provided a unique DB instance identifier for the Read Replica.
7. Verified that the source database was the primary RDS MySQL instance.
8. Configured the required DB instance and storage settings.
9. Configured the connectivity settings using the appropriate VPC and security group.
10. Reviewed the Read Replica configuration.
11. Selected **Create read replica**.
12. Waited for the Read Replica to complete provisioning.
13. Verified that the Read Replica status changed to **Available**.
14. Verified the **Replica Source** information to confirm that the replica was associated with the primary RDS database.
15. Captured screenshots as evidence of the successful Read Replica configuration.



Result

Successfully created an **Amazon RDS MySQL Read Replica** from the primary RDS database and verified that the replica was in **Available** status and associated with the correct source database.

Conclusion

An RDS Read Replica provides a separate database instance that asynchronously replicates data from the primary database and can be used to handle read traffic. This helps reduce the read workload on the primary database and improves application scalability.

